
PARA Ontology Workflow Documentation Rev 0.0.

June, 2025

Prepared by: Sara A. Medhat

Reviewed by: Majdi Saadeddine

Approved by: Faysal Shair

TABLE OF CONTENT

RELEVANT BACKGROUND INFORMATION AND PRE-REQUISITS	1
CHAPTER ONE: BRICKSCHEMA CSV STRUCTURE	4
CHAPTER TWO: EXTENDING BRICKSCHEMA	10
CHAPTER THREE: PROPERTIES AND DATA POINTS	12
COLLABORATORS	16
ONLINE DOCUMENTATION	16

Goal: To create a Building Ontology using BrickSchema for a building

RELEVANT BACKGROUND INFORMATION AND PRE-REQUISITS

An ontology is a shared language that lets us describe every element of a building model in a consistent, machine-readable way and finally shows how everything is connected.

In BrickSchema (V1.4), it specifies:

- What exists, and where it is.

By defining spatial hierarchy via `rec:isPartOf` or `rec:locatedIn`

ex.	<code>entity:BuildingXX-Name</code>	<code>rec:isPartOf</code>	<code>entity:Site-Name</code>
ex.	<code>entity:AHU-FloorXX-Count</code>	<code>rec:locatedIn</code>	<code>entity:BuildingXX-Name_Floor-XX</code>

- Which parts make up each entity

By listing components via `brick:hasPart`.

ex.	<code>entity:AHU-FloorXX-Count</code>	<code>brick:hasPart</code>	<code>entity:AHU-FloorXX-Count_CHW-Coil</code>
ex.	<code>entity:AHU-FloorXX-Count</code>	<code>brick:hasPart</code>	<code>entity:AHU-FloorXX-Count_SF</code>
ex.	<code>entity:AHU-FloorXX-Count_SF</code>	<code>brick:hasPart</code>	<code>entity:AHU-FloorXX-Count_SF_VFD</code>
OR			
ex.	<code>entity:AHU-FloorXX-Count_SF</code>	<code>brick:hasPart</code>	<code>entity:AHU-FloorXX-Count_SF_Motor</code>

- What system each element belongs to

By associating equipment with their system via `rec:isPartOf`.

ex.	<code>entity:AHU-FloorXX-Count</code>	<code>rec:isPartOf</code>	<code>entity:HVAC</code>
-----	---------------------------------------	---------------------------	--------------------------

- Which sensors or meters attach to which part: Connects meters/sensors as sub-parts.

ex.	<code>entity:AHU-FloorXX-Count_SF_VFD</code>	<code>brick:hasPart</code>	<code>entity:AHU-FloorXX-Count_SF_VFD_Electrical-Meter</code>
-----	--	----------------------------	---

- What points those meters measure

Links meters to their measured data via `brick:hasPoint`

ex.	<code>entity:AHU_SF_VFD_Electrical-Meter</code>	<code>brick:hasPoint</code>	<code>entity:AHU_SF_VFD_Electrical-Demand</code>
-----	---	-----------------------------	--

Using an ontology is the foundation for interoperable, scalable, and intelligent building operations that speed up integrations, and unlock the full potential of IoT and AI.

- Semantic Interoperability & Data Integration

Ontologies give every device, point, and system a precise meaning, enabling seamless data exchange.

- Scalability & Extensibility

Because ontologies are inherently open and extensible, you can add new classes (e.g., new sensor types) or properties (e.g., rated specifications as per equipment catalogues) without rewriting existing mappings. This flexibility is crucial for buildings evolving into digital twins.

- Automation, Analytics & AI Readiness

AI and advanced analytics thrive on well-structured data. An ontology ensures that every data point—be it temperature, CO₂, or occupancy—is semantically annotated, so machine learning pipelines can automatically discover, aggregate, and reason over building data.

- Consistency & Reuse of Best Practices

Established building ontologies (e.g., RealEstateCore, Brick, Haystack) embody industry best practices and naming conventions, preventing each project from reinventing the wheel.

 Goal: To create a Building Ontology using BrickSchema for a building

RELEVANT BACKGROUND INFORMATION AND PRE-REQUISITS

- Resource Description Framework “RDF”

RDF (Resource Description Framework) is the W3C’s standard model for representing information as a graph of simple, machine-readable statements called triples. RDF relies on Uniform Resource Identifiers (URIs) to name every resource in a triple. A Uniform Resource Identifier (URI) is a standardized string of characters that uniquely identifies an abstract or physical resource on the Web or in any information system. URIs form the backbone of RDF’s graph model by assigning each subject, predicate, and object a globally unique name that tools and machines can resolve or distinguish

- Ontology CSV Structure

Creating a BrickSchema CSV is easier when you understand two key ideas:

1. Graph: In RDF, a Graph (i.e., a collection of triples) organizes your data as connected statements, meaning the entire CSV forms a Graph.
2. Triples (Subject-Predicate-Object): Each row in your CSV becomes one triple :
 - a. Subject: What you are describing (e.g., a building or sensor).
 - b. Predicate: The relationship or attribute (e.g., brick:hasPoint).
 - c. Object: The destination of that relationship, which can be another entity or a value

With these basics, the next section shows exactly how to format your CSV, so every row is a clear, valid triple.

Every row in your CSV becomes one RDF triple. To ensure each triple is clear and machine-readable, your file should include these nine columns as shown below.

- Example: To state that the building "Dar Cairo" is part of the site "Smart Village", and give both a label

Column	Description
subject	The entity you are describing (e.g., entity:Dar-Cairo).
subjectType	The type of that resource (e.g., rec:Building).
predicate	The relationship or property (e.g., rec:isPartOf, brick:hasPart, rec:area).
object	The entity or value that the subject relates to (e.g., entity:Smart-Village for the rec:isPartOf relationship).
objectType	If the object is an entity, and its type is (e.g., rec:Site)
subject_prop_name	(Optional) A metadata property for the subject (e.g., rec:ArchitectureArea).
subject_prop_val	(Optional) The value for that metadata property (e.g., "2000 m2").
object_prop_name	(Optional) A metadata property for the object (e.g., brick:aggregationFunction)
object_prop_val	(Optional) The value for that metadata property (e.g., "Mean" or "Max").

Goal: To create a Building Ontology using BrickSchema for a building

RELEVANT BACKGROUND INFORMATION AND PRE-REQUISITS

PRE-REQUISIT

BIM Naming Convention

For all future BIM models, naming should follow this ontology-driven structure:

- **Site Name:**

ex: Smart-Village, Holborn-London, etc.

- **Building/Project Name:**

ex: Dar-Cairo, 150H, etc.

- **Floor/Level Names:**

Building-Name_Floor-XX

ex: 150H_Floor-7

- **Room Names:**

Building-Name_Floor-XX_Room-Name_Number

ex: 150H_Floor-7_Office_567

- **HVAC Zones:**

Building-Name_Floor-XX_HVAC-Zone

N.B. Room brick:isPartOf HVAC Zone

- **Parent Zones:**

Building-Name_Floor-XX_Parent-Zone

N.B. HVAC Zone brick:isPartOf Parent Zone

- **IFC_ID:**

Same as subject name without 'entity:' or spaces

N.B. Must be present for everything we want to showcase ex. on floors, rooms, zones, equipment.

- **Assets:**

Equipment-Name-Number

Equipment-Name-Number_Part-Name

Equipment-Name-Number_Part-Name_Point-Name

- **Systems:**

Must follow clear naming and connect visibly to corresponding assets.

BRICKSCHEMA CSV STRUCTURE

- **Spatial Details**

- Site

subject	subjectType	predicate	object	objectType
entity:Site-Name	rec:Site	brick:hasPart	entity:BuildingXX-Name	rec:Building
entity:Site-Name	rec:Site	brick:hasPart	entity:BuildingXX-Name	rec:Building

subject	subjectType	predicate	object	objectType
entity:Site-Name	rec:Site	brick:hasPoint	entity:chilledWaterRate	para:Rate
entity:Site-Name	rec:Site	brick:hasPoint	entity:egpToUsd	para:Rate
entity:Site-Name	rec:Site	brick:hasPoint	entity:electricityEmissionsRate	para:Rate
entity:Site-Name	rec:Site	brick:hasPoint	entity:electricityRate	para:Rate
entity:Site-Name	rec:Site	brick:hasPoint	entity:generationRate	para:Rate
entity:Site-Name	rec:Site	brick:hasPoint	entity:Smart-Village_COP	para:COP
entity:Site-Name	rec:Site	brick:hasPoint	entity:Smart-Village_Degree-Days	para:Degree_Days
entity:Site-Name	rec:Site	brick:hasPoint	entity:Smart-Village_Working-Day-Status	para:Working_Day_Status

CHAPTER ONE

BRICKSCHEMA CSV STRUCTURE

- **Spatial Details**

- Site

A ←

subject_prop_name	subject_prop_val	object_prop_name	object_prop_val	object_prop_name	object_prop_val
rdfs:label_en	Smart Village	rdfs:label_en	Chilled Water Rate	brick:hasUnit	para:EGP-PER-KiloW-HR
rdfs:label_en	Smart Village	rdfs:label_en	EGP to USD Exchange Rate	brick:hasUnit	para:USD-PER-EGP
rdfs:label_en	Smart Village	rdfs:label_en	Electricity Emissions Intensity	brick:hasUnit	para:KiloGM-CO2-PER-KiloW-HR
rdfs:label_en	Smart Village	rdfs:label_en	Electricity Rate	brick:hasUnit	para:EGP-PER-KiloW-HR
rdfs:label_en	Smart Village	rdfs:label_en	Generation Rate	brick:hasUnit	para:EGP-PER-KiloW-HR
rdfs:label_en	Smart Village	rdfs:label_en	Coefficient of Performance (COP)	brick:hasUnit	unit:UNITLESS
rdfs:label_en	Smart Village	rdfs:label_en	Degree Days	brick:hasUnit	para:CDD_DEGC_CALC
rdfs:label_en	Smart Village	rdfs:label_en	Working Day Status	brick:hasUnit	unit:UNITLESS

← A

Remarks:

1. The *rdfs:label_en* property is used to specify how an entity's name will appear in the front-end UI
2. When specifying a unit, you must choose from the QUDT units listed at <https://ontology.brickschema.org/qudt/Unit.html>, and for any other standard classes or properties, mainly listed under subject type or object type, use the BrickSchema namespace at <https://ontology.brickschema.org>.
3. If you define your own custom types or properties, don't prefix them with "brick:" or "rec:"; instead use the "para:" prefix

CHAPTER ONE

BRICKSCHEMA CSV STRUCTURE

• Spatial Details

- Building

subject	subjectType	predicate	object	objectType
entity:BuildingXX-Name	rec:Building	rec:isPartOf	entity:Site-Name	rec:Site
entity:BuildingXX-Name	rec:Building	rec:capacity	<blanknode>	<blanknode>
entity:BuildingXX-Name	rec:Building	rec:area	<blanknode>	<blanknode>

subject_prop_name	subject_prop_val	object_prop_name	object_prop_val
rdfs:label_en	Building Name	rdfs:label_en	Site Name
rdfs:label_en	Building Name	rec:seatingCapacity	X
rdfs:label_en	Building Name	rec:grossArea	Y

- In this example, we first declare that Building XX belongs to Site-Name — where “XX” is a placeholder you’ll replace with your actual project identifiers
- Next, we use blank nodes as arrays or “detail slots” to store the building’s seating capacity and area.
- Finally, we add a user-friendly label for display on the UI, which tells the front end to show “Building Name” and “Site Name”

Remark:

1. You can include extra *subject_prop_name/subject_prop_val* and *object_prop_name/object_prop_val* pairs within the same row to attach as many metadata attributes as needed — such as adding a *brick:hasUnit* for your *rec:grossArea*.
2. In cases where you’re only defining properties about the object (for example, specifying the unit), simply leave the *subject_prop_name* and *subject_prop_val* cells blank, and populate the *object_prop_name/object_prop_val* with your metadata.
3. When specifying a unit, you must choose from the QUDT units listed at <https://ontology.brickschema.org/qudt/Unit.html>

object_prop_name	object_prop_val
brick:hasUnit	unit:M2

BRICKSCHEMA CSV STRUCTURE

• Spatial Details

- Zones
 - Parent Zones

Parent Zone (def.) is a high-level spatial grouping of adjacent rooms or areas used for wayfinding, handover phases, or simplified documentation. It is not an HVAC Zone — its purpose is to organize the building's geometry

Subject	subjectType	predicate	object	objectType	subject_prop_name	subject_prop_val
entity:Zone-A	rec:Zone	rec:isPartOf	entity:Dar-Cairo	rec:Building	rdfs:label_en	Zone-A
entity:Zone-B	rec:Zone	rec:isPartOf	entity:Dar-Cairo	rec:Building	rdfs:label_en	Zone-B
entity:Zone-C	rec:Zone	rec:isPartOf	entity:Dar-Cairo	rec:Building	rdfs:label_en	Zone-C

- HVAC Zones

ASHRAE standards define an HVAC zone as a space or group of spaces within a building that is served by a single HVAC system or a portion of an HVAC system and is designed to have its temperature, humidity, and ventilation controlled to maintain a desired level of thermal comfort and air quality. Essentially, it's a defined area where the HVAC system provides consistent environmental conditions.

Subject	subjectType	predicate	object	objectType
entity:Dar-Cairo_Floor-1HC-H1	rec:HVACZone	rec:isPartOf	entity:Dar-Cairo_Floor-1	rec:Level
entity:Dar-Cairo_Floor-1LC-H1	rec:HVACZone	rec:isPartOf	entity:Dar-Cairo_Floor-1	rec:Level
entity:Dar-Cairo_Floor-1TB-H1	rec:HVACZone	rec:isPartOf	entity:Dar-Cairo_Floor-1	rec:Level

subject_prop_name	subject_prop_val
rdfs:label_en	1HC-H1
rdfs:label_en	1LC-H1
rdfs:label_en	1TB-H1

Remark:

1. You can also give any HVAC zone a front-end name by adding an `rdfs:label_en` pair on the same CSV row. Simply set `subject_prop_name` to `rdfs:label_en` and `subject_prop_val` to your desired display text, leaving the object's metadata cells blank because the label property is addressing the subject.
2. The same can be applied to lighting zones

BRICKSCHEMA CSV STRUCTURE

• Equipment Naming

Equipment should follow a structured format to associate it with a location and type.

Format:

<Equipment-Type>_<Floor-Number>_<Unique-Identifier_or_Count>

Examples:

Subject	subjectType	predicate	object
entity:CHWP-B1-1-PUMP-7-LEFT	brick:Chilled_Water_Booster_Pump	rec:locatedIn	entity:Dar-Cairo_Basement-1_MECH-RM-5_B125

objectType	subject_prop_name	subject_prop_val
rec:Room	rec:modelNumber	C-sc-L-EPSS-2A

In the above example, the triple states that the chilled-water booster pump identified as CHWP-B1-1-PUMP-7-LEFT is classified under brick:Chilled_Water_Booster_Pump and is located in room Dar-Cairo_Basement-1_MECH-RM-5_B125 (a rec:Room). The pump also carries a metadata property rec:modelNumber with the value "C-sc-L-EPSS-2A"

Subject	subjectType	predicate	object
entity:AHU-B1-02	brick:Air_Handling_Unit	rec:locatedIn	entity:Dar-Cairo_Basement-1

objectType	subject_prop_name	subject_prop_val
rec:Room	rec:modelNumber	39FX350

In the above example, the triple declares that the air-handling unit identified as AHU-B1-02 is classified under the Brick class brick:Air_Handling_Unit and is physically located in room Dar-Cairo_Basement-1 (a rec:Room). The same row also carries a metadata property rec:modelNumber with the value "39FX350"

BRICKSCHEMA CSV STRUCTURE

- **Equipment Naming**

Common Pitfalls:

- 1- Use abbreviations only if they are industry-standard (AHU, FCU, VAV, CRAC).
- 2- Both examples adhere to the prescribed naming convention:
 - a. CHWP-B1-1-PUMP-7-LEFT breaks down as:
 - i. Equipment-Type: CHWP (Chilled Water Booster Pump)
 - ii. Floor-Number: B1-1 (Basement Level 1)
 - iii. Unique-Identifier: PUMP-7-LEFT
 - b. AHU-B1-02 breaks down as:
 - i. Equipment-Type: AHU (Air Handling Unit)
 - ii. Floor-Number: B1 (Basement Level 1)
 - iii. Count: 02 (second unit on that floor)
- 3- Avoid Spaces: Do not include spaces anywhere in your entity names or type identifiers, spaces can only appear in the property value for the `rdfs:label_en` property value
- 4- Exact Case Matching: Brick and RDF type names are case-sensitive. For instance, `rec:Building` is not `rec:building` or `Rec:Building`.
- 5- Consistent Capitalization: Keep a consistent pattern to avoid confusion and duplication
- 6- Dashes (-) for Separating Words: Example: Use `dar-cairo` rather than `dar cairo` or `darCairo` when naming an entity.
- 7- Underscores (_) for Separating Segments: Use underscores to clearly demarcate different parts or segments of an identifier.
Example: `Dar-Cairo_Basement-3_Pump-Room_B331`

CHAPTER TWO

EXTENDING BRICKSCHEMA

Extending the Brick ontology allows you to tailor the schema to better represent the specific components within your building systems.

Brick provides a comprehensive set of predefined entity classes, such as Air Handling Units (AHUs) and Variable Air Volume (VAV) systems. Some of these classes have further subdivisions; for example, a chiller is a subclass under HVAC equipment.

```
brick:Entity
├── brick:Equipment
│   ├── brick:HVAC_Equipment
│   │   ├── brick:AHU
│   │   ├── brick:Air_Handler_Unit
│   │   ├── brick:Cooling_Tower
│   │   ├── brick:Chiller
│   │   ├── brick:Computer_Room_Air_Conditioning
│   │   ├── brick:Fan
│   │   │   ├── brick:Booster_Fan
│   │   │   ├── brick:Ceiling_Fan
│   │   │   ├── brick:Cooling_Tower_Fan
│   │   │   ├── brick:Discharge_Fan
│   │   │   ├── brick:Exhaust_Fan
│   │   │   │   ├── brick:Fume_Hood
│   │   │   ├── brick:Outside_Fan
│   │   │   ├── brick:Pressurization_Fan
│   │   │   └── brick:Return_Fan
```

- **Creating a New Subclass**

If your project requires a more specific classification not present in the existing Brick schema, you can define a new subclass. This process involves establishing a new class and specifying its relationship to an existing superclass, ensuring it inherits the appropriate properties and relationships.

- **Example: Defining a new custom HVAC component**

Suppose you need to introduce a new subclass called PIM "Pressure Independent Module" under the broader category of HVAC_Equipment. You would define it as follows:

subject	subjectType	predicate	object	subject_prop_name	subject_prop_val
para:Pressure_Independent_Module	owl:Class	rdfs:subClassOf	brick:Terminal_Unit	rdfs:label_en	Pressure Independent Module

Which will then be converted to Turtle (TTL) syntax to look like this:

```
para:Pressure_Independent_Module a owl:Class ;
    rdfs:subClassOf brick:Terminal_Unit ;
    rdfs:label "Pressure-Independent-Module"@en
```

EXTENDING BRICKSCHEMA

In the previous Turtle (TTL) syntax:

- brick:PIM is declared as a class using owl:Class.
- It is then specified as a subclass of brick:HVAC_Equipment using rdfs:subClassOf.
- By establishing this subclass relationship, PIM inherits all characteristics of HVAC_Equipment, ensuring consistency and integration within the Brick framework.

Remark:

- *It is crucial that the class is defined correctly, with the correct sub-class relationship outlined so that it inherits all properties from the super-class (HVAC_Equipment in this example).*
- *ALL additional sub-classes are saved in a separate PARA turtle file (.ttl) to be imported within any and all project ontology files. New sub-classes are reviewed first before addition to avoid redundancy and to ensure correct definition.*

CHAPTER THREE

PROPERTIES AND DATA POINTS

- **Structure of the CSV File and populating properties:**

subject	subjectType	predicate	object	objectType
entity:AHU-TYP2-F3_SF_Fan_VFD	brick:Fan_VFD	brick:ratedPowerInput	<blanknode>	<blanknode>
entity:AHU-TYP2-F3_SF_Fan_VFD	brick:Fan_VFD	brick:electricalPhaseCount	<blanknode>	<blanknode>
entity:AHU-TYP2-F3_SF_Fan_VFD	brick:Fan_VFD	brick:ratedVoltageInput	<blanknode>	<blanknode>
entity:AHU-TYP2-F3_CHW-Coil	brick:Chilled_Water_Coil	brick:coolingCapacity	<blanknode>	<blanknode>
entity:AHU-TYP2-F3_CHW-Coil	brick:Chilled_Water_Coil	para:ratedChilledWaterFlowrate	<blanknode>	<blanknode>

Each row declares one property of an entity using: subject, subjectType, predicate, object

- subject: The equipment or component (e.g., entity:AHU-TYP2-F3_SF_Fan_VFD)
- subjectType: Its Brick class (brick:Fan_VFD)
- predicate: The property name (e.g., brick:ratedPowerInput, brick:electricalPhaseCount, brick:coolingCapacity, or a custom parameter like para:ratedChilledWaterFlowrate)
- object: Always a blank node here (<blanknode>) to hold the actual value and unit
 - brick:value → the numeric literal
 - brick:hasUnit → the QUDT unit

object_prop_name	object_prop_val	object_prop_name	object_prop_val
brick:value	18.5	brick:hasUnit	unit:KiloW
brick:value	3	brick:hasUnit	unit:UNITLESS
brick:value	380	brick:hasUnit	unit:V
brick:value	159	brick:hasUnit	unit:KiloW
brick:value	5.44	brick:hasUnit	unit:L-PER-SEC

CHAPTER THREE

PROPERTIES AND DATA POINTS

- **Data Points**

- **Step 1: Identify the Parent Sensor/ Meter**

- Each data point must be linked to a parent entity, which is typically a piece of equipment or a sensor.
- In the below example, the parent entity is a virtual meter*, that is classified as a brick:Electrical_Meter.

**A virtual meter is a formula that uses data from existing physical meters to stand in for a device you don't actually have—no extra hardware needed. You define a rule in your engine, for example, "Building Electricity Demand = Sum of each individual floor electrical demand" set it to recalculate on a regular interval, and the system automatically pulls those real-time readings, runs the math, and displays the result just like a dedicated meter you never had to install.*

Subject	subjectType	predicate	object	objectType
entity:Dar-Cairo_Electrical-Virtual-Meter	brick:Electrical_Meter	brick:hasPoint	entity:Dar-Cairo_Electrical-Virtual-Meter-Avg-Elec-Demand	brick:Electric_Power_Sensor

subject_prop_name	subject_prop_val	object_prop_name	object_prop_val	object_prop_name	object_prop_val
		rdfs:label_en	Dar Cairo Avg Electricity Demand	brick:hasUnit	unit:KiloW

- **Step 2: Define the Data Point**

- Like in the above example; a data point represents the measured or calculated value in the case of a virtual meter. It is associated with a subject using the brick:hasPoint predicate.

- **Step 3: Add Metadata to the Data Point**

- Additional properties such as labels and units should be always specified for quantifiable points and properties:
 - The English label (human-readable name on the front-end) is added using
rdfs:label_en Dar Cairo Electricity Demand
 - The measurement unit is specified with
brick:hasUnit: unit:KiloW

- **Step 4: Linking to External References**

- Data points are linked to external time series or databases for real-time monitoring.
- In this case, the data point has an external reference:
ref:hasExternalReference <blanknode> ref:TimeseriesReference
- This means the total electrical demand is connected to a timeseries data base system.
- The external reference includes: Timeseries ID:
ref:hasTimeseriesId ELEC_KW_CALC
- Entity ID: This is a custom property that uniquely identifies the entity/device, that is assigned the addressed telemetry, in the external database system.
para:hasEntityId

CHAPTER THREE

PROPERTIES AND DATA POINTS

- **Data Points**

- **Step 4: Linking to External References (cont.)**

Subject	subjectType	predicate	object	objectType
entity:Dar-Cairo_Electrical-Virtual-Meter_Total-Elec-Demand	brick:Electric_Power_Sensor	ref:hasExternalReference	<blanknode>	ref:TimeseriesReference

subject_prop_name	subject_prop_val	object_prop_name	object_prop_val	object_prop_name	object_prop_val
		ref:hasTimeseriesId	ELEC_KW_CALC	para:hasEntityId	Smart Village

- **Blank Nodes:**

When modeling data, blank nodes can function similarly to arrays or storage locations, grouping related properties without the need for explicit identifiers. The object of the property relation (e.g, brick:ratedPowerInput) becomes a blank node, we assign the related properties (e.g., brick:hasUnit, brick:value) to the blanknode (a placeholder).

Subject	subjectType	predicate	object	objectType
entity:AHU-B1-01_SF_Motor	brick:Motor	brick:ratedPowerInput	<blanknode>	<blanknode>

subject_prop_name	subject_prop_val	object_prop_name	object_prop_val	object_prop_name	object_prop_val
		brick:value	5.5	brick:hasUnit	unit:KiloW

Remarks

- This CSV format is going to appear when converted to TTL as follows, which translates to "supply fan has a rated power input of 5.5 kW, and similarly the cooling coil has a cooling capacity of 50.0 kW."
- These are properties not data points, yet they are also associated with blank nodes that encapsulate the value and unit.

```
_:supplyFan a brick:Supply_Fan ;
  brick:ratedPowerInput [
    brick:value 5.5 ;
    brick:hasUnit unit:KiloWatt
  ] .

_:coolingCoil a brick:Cooling_Coil ;
  brick:coolingCapacity [
    brick:value 50.0 ;
    brick:hasUnit unit:KiloWatt
  ] .
```


PROPERTIES AND DATA POINTS

• Aggregations

Aggregations summarize multiple time-series readings into a single value using a specified function (e.g., mean, max). If in the external time-series database these aggregations have a TimeseriesId (telemetry names) that is different from the original data point then a new data point needs to be modeled. This new data point will typically have the same data point class/type. The aggregation property is applied to data points, and provides applications a way to distinguish between multiple points on an entity which all have the same data type.

In BrickSchema, you declare an aggregation by linking the sensor or virtual meter data point to a blank node via `brick:aggregate`, then specify the function with `brick:aggregationFunction`.

Subject	subjectType	predicate	object	objectType
entity:Dar-Cairo_Electrical-Virtual-Meter_Avg-Elec-Demand	brick:Electric_Power_Sensor	brick:aggregate	<blanknode>	<blanknode>

subject_prop_name	subject_prop_val	object_prop_name	object_prop_val
		brick:aggregationFunction	mean

In the aggregation example above:

- `brick:aggregationFunction` is a property that defines how to summarize the readings.
- The value you give it—such as `mean` or `max`—is the value of the object of that triple, specifying the aggregation method.

So, in the above example:

- `brick:aggregationFunction` names the property/ function
- “`mean`” is the literal function telling you to compute the average (mean) of the instantaneous power-demand measurement on the building level from the virtual meter, and store that single number

Which will then appear in TTL form like this

```
entity:Dar-Cairo_Electrical-Virtual-Meter_Avg-Elec-Demand a brick:Electric_Power_Sensor ;
    brick:aggregate [
        brick:aggregationFunction "mean"
    ] ;
```

Collaborators In alphabetical order

Currently active

- Majdi Saadedine
- Nada Ebrahim Badawy
- Omar Shair

Former collaborator

Omar El Refaie

Online Documentation

[Introduction — Brick Ontology Documentation](#)

Brick is an open-source effort to standardize semantic descriptions of the physical, logical and virtual assets in buildings and the relationships between them. Brick consists of an extensible dictionary of terms and concepts in and around buildings, a set of relationships for linking and composing concepts together, and a flexible data model permitting seamless integration of...

[Explorer](#) ontology.brickschema.org

This website uses heuristics to determine the schema of RDF vocabulary. This process is not perfect and may not always provide an accurate view of the underlying RDF data. Users are advised to use the output of this website as a guide only and to check the accuracy of the results against the original Turtle files.

(This is where you see how any entity can be defined and what classes already exist)

[GitHub - BrickSchema/docs: Online developer-focused documentation for the use of Brick.](#)

Online developer-focused documentation for the use of Brick. Hosted at <https://docs.brickschema.org> - BrickSchema/docs

 GitHub

[BrickSchema Documentation — brickschema documentation](#)

The brickschema package makes it easy to get started with Brick and Python. Among the features it provides are:

 brickschema.readthedocs.io

Virtual Building Information System (VBIS) Support:

[VBIS in Action — VBIS](#)

The video showcases an example of the virtual walk through linked for the site linked with the site document management system. The user can virtually identify the asset and then click on the asset to search and locate documents related to the asset hosted in the document system.

 VBIS

[SBOS Playground](#)

PlayGround is a safe building operating system (SBOS), that incorporates a structured semantic representation of the building (based on Brick) to inform the safe, multi-tenant execution of untrusted applications. We use the semantic representation to implement (a) a novel graph-based capability mechanism for fine-grained and expressive access control management, and...

 brickschema.github.io

Online Documentation (cont.)

External References (ref-schema): Timeseries DB IDs, BACnet, MODBUS, IFC

Documentation: [Ref Schema \(brickschema.org\)](#)

GitHub: [ref-schema/ at main · gtfierro/ref-schema · GitHub](#)

Examples (BACnet and Timeseries): [External Representation – Brick Ontology Documentation \(brickschema.org\)](#)

Brick Schema Whitepaper:

[Brick_Schema_Whitepaper.pdf \(memoori.com\)](#)

Playground: A Safe Building Operating System:

[playground2024fu.pdf \(gtf.fyi\)](#) [Playground: A Safe Building Operating System | IEEE Conference Publication | IEEE Xplore](#)

YASGUI – SPARQL web GUI: [Yasgui - Triply](#)



[GitHub - TriplyDB/Yasgui: Yet Another Sparql GUI](#)

Yet Another Sparql GUI. Contribute to TriplyDB/Yasgui development by creating an account on GitHub.